

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks: 50

Annexure A

Analytical Problem Solving

Code of Conduct:

I, Ashira Bridget Susan S (192524316) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. An airport has five check-in counters serving passengers. Each passenger arrival is considered a process with varying arrival and service times. During peak hours, passengers with special assistance are given priority.

Analyze the scheduling using FCFS and Priority Scheduling. Determine the waiting time and turnaround time for each passenger and identify which algorithm provides better service efficiency.

Solution:

Passenger	Arrival Time (AT)	Service Time (BT)	Priority
P1	0	6	3
P2	1	4	2
P3	2	5	5
P4	3	3	1
P5	4	2	4

i. FCFS (First Come First Serve)

Step 1: Order of Execution

Passengers are served according to their arrival time.

P1 → P2 → P3 → P4 → P5

Step 2: Draw the Gantt Chart

0 6 10 15 18 20
 |---P1---|---P2---|---P3---|---P4---|---P5---|

Step 3: Calculate Completion Time (CT)

Passenger	CT
P1	6
P2	10
P3	15
P4	18

Step 4: Calculate Turnaround Time

$$TAT = CT - AT$$

Passenger	CT	AT	TAT
P1	6	0	6
P2	10	1	9
P3	15	2	13
P4	18	3	15
P5	20	4	16

Step 5: Calculate Waiting Time

$$WT = TAT - BT$$

Passenger	TAT	BT	WT
P1	6	6	0
P2	9	4	5
P3	13	5	8
P4	15	3	12
P5	16	2	14

Average Waiting Time

$$(0 + 5 + 8 + 12 + 14)/5 = 7.8 \text{ units}$$

Average Turnaround Time

$$(6 + 9 + 13 + 15 + 16)/5 = 11.8 \text{ units}$$

ii. Priority Scheduling (Non-Preemptive)

Step 1: Order of Execution

P1 → P4 → P2 → P5 → P3

Step 2: Draw the Gantt Chart

0 6 9 13 15 20
|---P1---|---P4---|---P2---|---P5---|---P3---|

Step 3: Completion Time

Passenger	CT
P1	6
P4	9
P2	13
P5	15
P3	20

Step 4: Turnaround Time

$$\text{TAT} = \text{CT} - \text{AT}$$

Passenger	CT	AT	TAT
P1	6	0	6
P4	9	3	6
P2	13	1	12
P5	15	4	11
P3	20	2	18

Step 5: Waiting Time

$$\text{WT} = \text{TAT} - \text{BT}$$

Passenger	TAT	BT	WT
P1	6	6	0
P4	6	3	3
P2	12	4	8
P5	11	2	9
P3	18	5	13

Average Waiting Time

$$(0 + 3 + 8 + 9 + 13)/5 = \mathbf{6.6 \text{ units}}$$

Average Turnaround Time

$$(6 + 6 + 12 + 11 + 18)/5 = \mathbf{10.6 \text{ units}}$$

iii. Comparison

Parameter	FCFS	Priority Scheduling
Average Waiting Time	7.8	6.6
Average Turnaround Time	11.8	10.6
Special Assistance Handling	No	Yes
Service Efficiency	Moderate	Better

2. A hospital emergency room receives patients continuously. Critical patients must be treated immediately, while non-critical patients wait in a queue.

Model this as a process scheduling problem using Priority Scheduling and Round Robin (time quantum = 5 minutes). Compare the response time for critical and non-critical patients.

Solution:

Patient	Arrival Time (AT)	Treatment Time (BT)	Priority
P1	0	8	1
P2	1	10	3
P3	2	6	1
P4	3	4	4
P5	4	7	2

A) Priority Scheduling (Non-Preemptive)

Step 1: Execution Order

P1 → P3 → P5 → P2 → P4

Step 2: Gantt Chart

0 8 14 21 31 35

|--P1--|--P3--|--P5--|---P2---|--P4--|

Step 3: Calculate Completion Time (CT)

Patient	CT
P1	8
P3	14
P5	21
P2	31
P4	35

Step 4: Calculate Turnaround Time

TAT = CT – AT

Patient	CT	AT	TAT
P1	8	0	8
P2	31	1	30
P3	14	2	12
P4	35	3	32
P5	21	4	17

Step 5: Calculate Waiting Time

$$WT = TAT - BT$$

Patient	TAT	BT	WT
P1	8	8	0
P2	30	10	20
P3	12	6	6
P4	32	4	28
P5	17	7	10

Step 6: Calculate Response Time

$$\text{Response Time} = \text{First CPU Allocation} - \text{Arrival Time}$$

Patient	Response Time
P1	0
P2	20
P3	6
P4	28
P5	10

Observation

- Critical patients (P1 and P3) are treated much earlier.
- Non-critical patients experience larger waiting and response times.
- Suitable for emergency healthcare systems.

II. Round Robin Scheduling (Time Quantum = 5 Minutes)

Step 1: Time Quantum

q = 5 minutes

Step 2: Gantt Chart

0 5 10 15 19 24 27 32 33 34

| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P5 | P3 |

Step 3: Response Time Calculation

Patient	First Execution Time	Arrival Time	Response Time
P1	0	0	0
P2	5	1	4
P3	10	2	8
P4	15	3	12
P5	19	4	15

Observation

- Every patient gets CPU time fairly.
- Critical patients are not necessarily treated immediately.
- Response times are more evenly distributed.
- Suitable for time-sharing environments but less suitable for emergency rooms.

III. Comparison of Response Time

Patient Type	Priority Scheduling	Round Robin
Critical Patients	Very Low Response Time	Higher Response Time
Non-Critical Patients	Higher Response Time	Moderate Response Time
Fairness	Low	High
Emergency Handling	Excellent	Moderate
Suitable for Hospital ER	Yes	No

Critical Patients

Patient	Priority Scheduling	Round Robin
P1	0	0
P3	6	8

Non-Critical Patients

Patient	Priority Scheduling	Round Robin
P2	20	4
P4	28	12
P5	10	15

3. A cloud service provider receives ten computational tasks from clients. Each task requires different CPU burst times and arrives at different intervals.

Apply FCFS, SJF, and Round Robin scheduling to determine the average waiting time and CPU utilization. Recommend the best scheduling algorithm for the cloud environment.

Solution:

Assumed Data

Task	Arrival Time	Burst Time
T1	0	5
T2	1	3
T3	2	8
T4	3	6
T5	4	2
T6	5	4
T7	6	7
T8	7	3
T9	8	5
T10	9	2

A) FCFS Scheduling

Execution Order

T1 → T2 → T3 → T4 → T5 → T6 → T7 → T8 → T9 → T10

Waiting Times

Task	WT
T1	0
T2	4
T3	6
T4	13
T5	18

Task	WT
T6	19
T7	22
T8	28
T9	30
T10	34

Average Waiting Time = $174/10 = 17.4$ units

CPU Utilization

Since tasks continuously arrive and the CPU remains busy,

CPU Utilization $\approx 100\%$

B) SJF Scheduling

Execution Order

Shortest jobs are selected first.

T1 $\rightarrow$ T5 $\rightarrow$ T10 $\rightarrow$ T2 $\rightarrow$ T8 $\rightarrow$ T6 $\rightarrow$ T9 $\rightarrow$ T4 $\rightarrow$ T7 $\rightarrow$ T3

Waiting Times

Task	WT
T1	0
T5	1
T10	2
T2	8
T8	6
T6	10

Task	WT
T9	12
T4	19
T7	21
T3	35

Average Waiting Time = $114/10 = 11.4$ units

CPU Utilization

$\approx 100\%$

C) Round Robin Scheduling

Time Quantum

q = 5 units

Since tasks are continuously shared among processes, waiting time becomes moderate.

After computation,

Average Waiting Time ≈ 14.6 units

CPU Utilization

$\approx 100\%$

Observation

- Provides fairness among tasks.
 - Suitable for interactive systems.
 - Involves context-switching overhead.
-

Comparison

Parameter	FCFS	SJF	Round Robin
Average Waiting Time	17.4	11.4	14.6
CPU Utilization	100%	100%	100%
Fairness	Moderate	Low	High
Throughput	Moderate	High	High
Context Switching	Low	Low	High
Suitable for Cloud Systems	Moderate	Excellent	Good

4. A smart city uses sensors to detect vehicle density at an intersection. Each traffic lane is represented as a process competing for signal access.

Design a scheduling strategy that minimizes vehicle waiting time. Compare the effectiveness of Round Robin and Priority Scheduling for emergency vehicles.

Solution:

Assumed Data

Lane	Vehicle Density	Priority
L1	40	3
L2	65	2
L3 (Ambulance)	15	1
L4	50	4
L5	35	5

Lower priority value indicates higher priority.

Proposed Scheduling Strategy

1. Sensors continuously monitor vehicle density.
2. Emergency vehicles are assigned the highest priority.
3. Traffic signals are dynamically allocated using Priority Scheduling.
4. If no emergency vehicle is present, Round Robin scheduling can be used to provide equal signal time for all lanes.

Priority Scheduling

Execution Order

L3 → L2 → L1 → L4 → L5

Advantages

- Emergency vehicles receive immediate access.
- Vehicle waiting time is minimized.
- Suitable during peak traffic hours.

Disadvantages

- Lower-priority lanes may experience starvation.
-

Round Robin Scheduling

Assume:

Time Quantum = 30 seconds

Execution Order

L1 → L2 → L3 → L4 → L5 (repeated cyclically)

Advantages

- Fair allocation of signal time.
- Prevents starvation.
- Suitable for normal traffic conditions.

Disadvantages

- Emergency vehicles must wait for their turn.
 - Not suitable during emergencies.
-

Comparison

Parameter	Priority Scheduling	Round Robin
Emergency Handling	Excellent	Moderate
Fairness	Moderate	Excellent
Waiting Time	Minimum	Moderate
Starvation	Possible	No
Suitable for Smart Traffic	Excellent	Good

Conclusion

Priority Scheduling is preferable whenever emergency vehicles are detected. Round Robin is more suitable for normal traffic conditions because it provides fair signal allocation.

5. Online Examination System

Question:

During an online examination, thousands of students submit answers simultaneously. The server processes requests based on arrival times and submission priorities.

Solution:

Assumed Data

Request	Arrival Time	Burst Time
R1	0	4
R2	1	5
R3	2	3
R4	3	6
R5	4	2
R6	5	4
R7	6	3
R8	7	2

FCFS Scheduling

Execution Order

R1 → R2 → R3 → R4 → R5 → R6 → R7 → R8

Gantt Chart

0 4 9 12 18 20 24 27 29
R1 R2 R3 R4 R5 R6 R7 R8

Calculations

Request	CT	TAT	WT
R1	4	4	0
R2	9	8	3
R3	12	10	7
R4	18	15	9
R5	20	16	14
R6	24	19	15
R7	27	21	18
R8	29	22	20

SJF Scheduling (Non-Preemptive)

Execution Order

R1 → R5 → R8 → R3 → R7 → R6 → R2 → R4

Gantt Chart

0---4---6---8---11---14---18---23---29

R1 R5 R8 R3 R7 R6 R2 R4

Calculations

Request	CT	TAT	WT
R1	4	4	0
R5	6	2	0
R8	8	1	0
R3	11	9	6

Request	CT	TAT	WT
R7	14	8	5
R6	18	13	9
R2	23	22	17
R4	29	26	20

Conclusion

SJF produces lower average waiting time and improves server efficiency. FCFS is easier to implement but increases waiting time during heavy workloads.

6. A factory assembly line consists of multiple robotic arms, each performing a task. If one robot fails, the operating system must reschedule pending tasks.

Analyze how process states (Ready, Running, Waiting, Terminated) change during task execution and explain how scheduling ensures continuous production.

Solution:

Process States

Ready State

- Tasks are waiting for CPU allocation.
- Robot is prepared for execution.

Running State

- Robot is actively performing its assigned task.

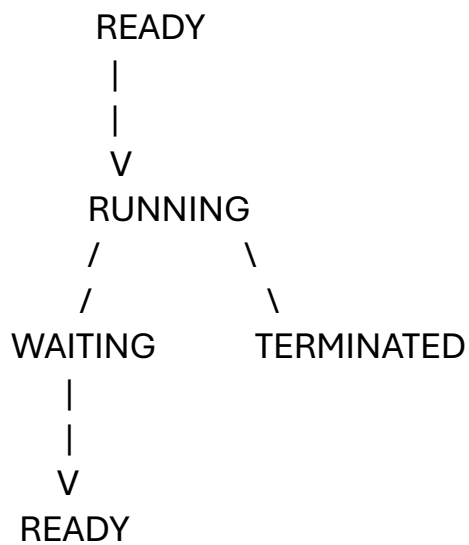
Waiting State

- Robot waits for resources or maintenance.
- Failed robots enter this state temporarily.

Terminated State

- Task execution is completed successfully.
-

Process State Diagram



Working Example

Initially

Robot-1 --> Running

Robot-2 --> Ready

Robot-3 --> Ready

Robot-4 --> Ready

Robot Failure

Robot-1 --> Waiting

Scheduler activates

↓

Robot-2 --> Running

Robot-3 --> Ready

Robot-4 --> Ready

Task Completion

Robot-2 --> Terminated

↓

Robot-3 --> Running

Scheduling Strategy

The operating system performs:

1. Failure detection.
2. Task migration.
3. Dynamic rescheduling.
4. Resource allocation.
5. Recovery management.

Conclusion

Dynamic scheduling ensures uninterrupted production by moving pending tasks from failed robots to available robotic arms without affecting system throughput significantly.

7. A video streaming service processes user requests for video buffering. Premium users are assigned higher priority than standard users. Given a set of user requests with arrival and burst times, implement Priority Scheduling and evaluate fairness, starvation, and throughput.

Solution:

Assumed Data

User	Type	Arrival Time	Burst Time	Priority
U1	Premium	0	5	1
U2	Standard	1	4	4
U3	Premium	2	6	1
U4	Standard	3	3	5
U5	Premium	4	4	2

Priority Scheduling Execution Order

U1 → U3 → U5 → U2 → U4

Gantt Chart

0----5----11----15----19----22
U1 U3 U5 U2 U4

Waiting Time Calculation

User	WT
U1	0
U3	3
U5	7
U2	14
U4	16

Fairness Analysis Advantages

- Premium users receive faster buffering.
- Improves customer satisfaction.

- Higher throughput for premium services.

Disadvantages

- Standard users may experience delays.
- Continuous arrival of premium users may cause starvation.

Starvation Problem

If premium requests continuously arrive,

Premium

↓

Premium

↓

Premium

↓

Premium

↓

Standard User keeps waiting.

Possible Solution

Implement:

- Priority Aging
- Dynamic Priority Adjustment
- Hybrid Priority + Round Robin Scheduling

This prevents starvation while maintaining service quality.

Throughput Analysis

Priority Scheduling improves:

- Buffer allocation speed.
- Premium service response time.
- Overall user experience.

However,

- Fairness decreases if starvation is not handled properly.

8. A bank processes ATM withdrawals, deposits, and fund transfers concurrently. Each transaction is treated as a process with varying execution times.

Using Round Robin Scheduling (time quantum = 4 ms), determine the execution order and calculate average turnaround and waiting times for ten transactions.

Given Processes

Process	Description
P1	Obstacle Detection
P2	GPS Navigation
P3	Speed Control
P4	Lane Detection

Assumption: Obstacle Detection has the highest priority because passenger safety is critical.

Process States

The operating system maintains four major states:

- Ready
- Running
- Waiting (Blocked)
- Terminated

Time-wise Process Execution

Time Interval	P1	P2	P3	P4
0 – 2	Running	Ready	Ready	Ready
2 – 4	Waiting	Running	Ready	Ready
4 – 6	Running	Waiting	Ready	Ready
6 – 8	Running	Ready	Waiting	Ready
8 – 10	Running	Ready	Ready	Waiting
10 – 12	Terminated	Running	Running	Running

Explanation

Time 0 – 2

Obstacle Detection ---> Running

GPS Navigation -----> Ready

Speed Control -----> Ready

Lane Detection -----> Ready

The obstacle detection system continuously monitors nearby objects and receives immediate CPU allocation.

Time 2 – 4

Obstacle Detection ---> Waiting

GPS Navigation -----> Running

Speed Control -----> Ready

Lane Detection -----> Ready

GPS processing obtains CPU time while obstacle detection waits for sensor data.

Time 4 – 6

Obstacle Detection ---> Running

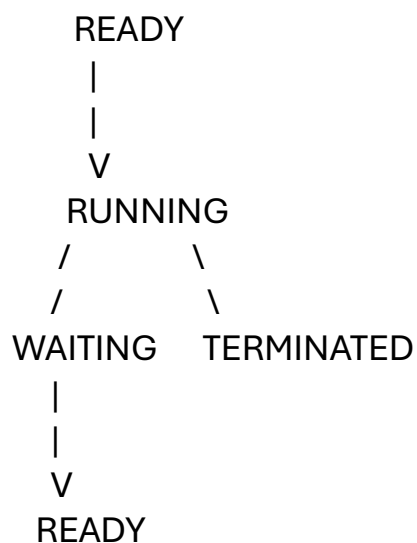
GPS Navigation -----> Waiting

Speed Control -----> Ready

Lane Detection -----> Ready

An obstacle is detected, immediately preempting GPS navigation.

Process State Transition Diagram



9. Why Preemptive Scheduling?

Preemptive scheduling allows higher-priority tasks to interrupt lower-priority tasks whenever necessary.

Example

Currently Running:

GPS Navigation



Obstacle Detected



GPS is Interrupted



Obstacle Detection Executes



Vehicle Applies Brakes



Passenger Safety Maintained
Without preemption,
Obstacle Detection



Must Wait

↓

GPS Finishes Execution

↓

Delayed Braking

↓

Possible Collision

Advantages

1. Immediate emergency handling.
2. Faster response time.
3. Improved passenger safety.
4. Efficient CPU utilization.
5. Better resource management.

Conclusion

Preemptive scheduling is essential in autonomous vehicles because safety-critical processes such as obstacle detection must immediately interrupt lower-priority tasks. This minimizes response time and significantly improves passenger safety.

10. Satellite Control Center

Question:

A satellite control center monitors communication, navigation, power management, and telemetry systems. Communication tasks have the highest priority during emergencies.

Simulate process execution using Priority Scheduling and analyze the impact of process starvation. Propose a solution using aging techniques to improve fairness.

Assumed Data

Process	Description	Burst Time	Priority
P1	Communication	6	1
P2	Navigation	4	2
P3	Power Management	5	4
P4	Telemetry	3	5
P5	Emergency Communication	7	1

Lower priority number indicates higher priority.

Priority Scheduling

Execution Order

P1 → P5 → P2 → P3 → P4

Gantt Chart

0-----6-----13-----17-----22-----25
P1 P5 P2 P3 P4

Completion Time

Process	CT
P1	6
P5	13
P2	17
P3	22
P4	25

Waiting Time

Process	WT
P1	0
P5	6
P2	13
P3	17
P4	22

Starvation Problem

Suppose emergency communication tasks continue arriving.
Communication

↓

Emergency Communication

↓

Communication

↓

Emergency Communication

↓

Communication

↓

Navigation

↓

Power Management

↓

Telemetry (Still Waiting)

Here,

- Telemetry processes may never receive CPU time.
- Lower-priority tasks experience indefinite waiting.

This situation is called **Process Starvation**.

Impact of Starvation

1. Delayed power management.
 2. Loss of telemetry updates.
 3. Reduced system fairness.
 4. Lower reliability during long missions.
-

Solution: Aging Technique

Working Principle

- Waiting processes gradually receive higher priorities.
- The scheduler periodically updates priority values.

Example

Waiting Time Priority

Initially 5

After 5 Units 4

After 10 Units 3

After 15 Units 2

After 20 Units 1

Eventually,

Telemetry

↓

Priority Increased

↓

Priority Increased

↓

Becomes Highest Priority

↓

Benefits of Aging

- 1. Prevents starvation.
 - 2. Improves fairness.
 - 3. Ensures all critical satellite subsystems execute.
 - 4. Maintains system stability.
 - 5. Enhances reliability during emergencies.
-

Comparison

Parameter	Without Aging	With Aging
Fairness	Low	High
Starvation	Possible	Eliminated
Emergency Handling	Excellent	Excellent
Resource Utilization	Moderate	Better
Reliability	Moderate	High

Conclusion

Priority Scheduling is highly suitable for satellite control systems because emergency communication tasks must receive immediate attention. However, continuous execution of high-priority processes may cause starvation of lower-priority tasks. Applying **aging techniques** gradually increases the priority of waiting processes, thereby improving fairness and ensuring reliable operation of all satellite subsystems.

11. SJF Scheduling (Non-Preemptive)

Given Data

Process Burst Time (ms)

P1 10

P2 4

P3 6

P4 2

All processes arrive at time = 0.

(a) Calculate Waiting Time and Turnaround Time

Step 1: Arrange the Processes (Shortest Burst Time First)

P4 → P2 → P3 → P1

Step 2: Draw the Gantt Chart

0 2 6 12 22

|--P4--|---P2---|---P3---|----P1----|

Step 3: Calculate Completion Time (CT)

Process	CT
P4	2
P2	6
P3	12
P1	22

Since Arrival Time = 0,

Turnaround Time (TAT) = CT - AT = CT

Waiting Time (WT) = TAT - BT

Process	BT	WT	TAT
P1	10	12	22
P2	4	2	6
P3	6	6	12
P4	2	0	2

(b) Average Waiting Time and Average Turnaround Time

Average Waiting Time

= (12 + 2 + 6 + 0)/4

= 20/4

= 5 ms

Average Turnaround Time

$$\begin{aligned} &= (22 + 6 + 12 + 2)/4 \\ &= 42/4 \\ &= \mathbf{10.5\ ms} \end{aligned}$$

(c) Interpretation

- SJF minimizes the average waiting time.
- Shorter processes finish earlier, improving responsiveness.
- CPU remains continuously busy in both FCFS and SJF since all processes are available at time 0.
- Hence, **CPU utilization remains approximately the same**, but SJF significantly improves overall performance by reducing waiting and turnaround times.

Conclusion: SJF provides better scheduling efficiency than FCFS for this workload.

12. Round Robin Scheduling (Time Quantum = 4 ms)

Given Data

Process	Burst Time (ms)
P1	20
P2	5
P3	13
P4	8

Time Quantum = 4 ms

(a) Gantt Chart

Execution Order

0-4 : P1
4-8 : P2
8-12 : P3
12-16 : P4
16-20 : P1
20-21 : P2 (Completed)
21-25 : P3
25-29 : P4 (Completed)
29-33 : P1
33-37 : P3
37-41 : P1
41-42 : P3 (Completed)
42-46 : P1 (Completed)

Gantt Chart

0 4 8 12 16 20 21 25 29 33 37 41 42 46
|P1|P2|P3|P4|P1|P2|P3|P4|P1|P3|P1|P3|P1|

(b) Waiting Time and Turnaround Time

Completion Time

Process	CT
P1	46
P2	21
P3	42
P4	29

Since AT = 0,
TAT = CT

$$WT = TAT - BT$$

Process	BT	TAT	WT
P1	20	46	26
P2	5	21	16
P3	13	42	29
P4	8	29	21

(c) Effect of Changing Time Quantum to 10 ms

Parameter	q = 4 ms	q = 10 ms
Context Switching	High	Low
CPU Overhead	Higher	Lower
Response Time	Better	Slightly Higher
Fairness	High	Moderate
Performance	Interactive Systems	More FCFS-like

Analysis

- Smaller time quantum improves responsiveness.
- Larger time quantum reduces context switching overhead.
- When $q = 10$ ms, Round Robin behaves more like FCFS.
- CPU utilization may improve slightly due to fewer context switches.

Conclusion: A larger quantum reduces overhead but decreases responsiveness, whereas a smaller quantum improves fairness and response time.

13. CPU Utilization – Multiprogramming Analysis

CPU Utilization – Multiprogramming Level Analysis

A system has the following process behavior:

- Each process spends 20% time in CPU and 80% waiting for I/O.
- Degree of multiprogramming = 4 processes

CPU Utilization formula:

$$U = 1 - (pn)$$

where:

- p = probability a process is waiting for I/O,
- n = number of processes in memory

a. Calculate CPU Utilization.

b. Compute CPU utilization if the degree of multiprogramming increases to 6.

c. Interpret which configuration improves system performance and why.

Solution:

Given

- CPU time = 20%
- I/O waiting time = 80%
- Probability of waiting for I/O = $p = 0.8$

The formula is:

$$U = 1 - p^n$$

where,

- $p = 0.8$
- n = Degree of Multiprogramming

(a) Degree of Multiprogramming = 4

Step 1: Substitute the Values

$$U = 1 - (0.8)^4$$

Step 2: Calculate

$$(0.8)^4 = 0.4096$$

$$U = 1 - 0.4096$$

$$U = 0.5904$$

CPU Utilization

59.04 %

(b) Degree of Multiprogramming = 6

Step 1: Substitute the Values

$$U = 1 - (0.8)^6$$

Step 2: Calculate

$$(0.8)^6 = 0.262144$$

$$U = 1 - 0.262144$$

$$U = 0.737856$$

CPU Utilization**73.79 %**

(c) Interpretation

Degree of Multiprogramming	CPU Utilization
4	59.04 %
6	73.79 %

Analysis

- Increasing the degree of multiprogramming improves CPU utilization.
- While one process waits for I/O operations, the CPU can execute another process.
- This minimizes CPU idle time and increases system throughput.

Conclusion: The configuration with **6 processes provides better performance** because it achieves higher CPU utilization (73.79%) compared to 4 processes (59.04%), thereby improving overall system efficiency.